

CAHIER DES CHARGES

BiblioTech

Recommandation intelligente de livres



Sommaire

- 1. Contexte
- 2. Descriptif de la demande
- 3. Architecture technique
- 4. Contraintes, limites et critères d'acceptation
- 5. Annexes

1. Contexte

1.1. Présentation des parties

Prestataire : Myriam Kessali, développeuse.

Client : Bibliothèque nationale de France (client fictif dans le cadre du projet BiblioTech).

1.2. Présentation du projet

Dans un contexte de transformation numérique et d'augmentation des contenus culturels disponibles en ligne, BiblioTech vise à faciliter la découverte de livres grâce à une application de recherche et de recommandation.

Le projet exploite le dataset Book-Crossing, composé de métadonnées de livres, de profils utilisateurs et de notes. Il met en place une chaîne complète allant de l'ingestion des données brutes jusqu'à une interface interactive, en passant par le nettoyage, le stockage, l'analyse, l'entraînement d'un modèle de recommandation et l'exposition des fonctionnalités via une API.

Une distinction est faite entre l'ISBN, qui représente une édition, et l'œuvre logique. BiblioTech utilise pour cela un identifiant book_key construit à partir du titre et de l'auteur normalisés (exemple : 1984|george orwell). Le système de recommandation travaille principalement au niveau de l'œuvre.

2. Descriptif de la demande

2.1. Objectifs

- Collecter et centraliser les données du dataset Book-Crossing.
- Nettoyer, normaliser et contrôler la qualité des données.
- Organiser les données dans une architecture Bronze / Silver / Gold sur MinIO.
- Charger les données utiles dans PostgreSQL.
- Construire des indicateurs analytiques, notamment la popularité des œuvres.
- Construire un système de recommandation collaborative item-based.
- Permettre la recherche de livres par titre ou auteur.
- Permettre la recommandation à partir d'une œuvre et la recommandation personnalisée à partir des notes d'un utilisateur applicatif.
- Exposer les fonctionnalités via FastAPI.
- Fournir une interface utilisateur Streamlit avec métadonnées, couvertures et recommandations.

2.2. FP1 - Gestion et préparation des données

Cette fonction couvre la collecte, le nettoyage, la transformation, le contrôle qualité et le stockage des données.

Critère	Niveau attendu	Flexibilité / évolution
Sources	Dataset Book-Crossing : livres, utilisateurs et ratings	Extension possible vers Open Library / Google Books
Formats	CSV en Bronze ; Parquet en Silver et Gold	Formats supplémentaires possibles
Nettoyage	Encodage, chaînes, doublons, années invalides, ratings implicites et incohérences	Règles ajustables
Identité des œuvres	Création de book_key à partir du titre et de l'auteur normalisés	Amélioration future de la résolution des œuvres
Contrôle qualité	Vérification des références utilisateurs/livres et suppression des données orphelines	Contrôles supplémentaires possibles
Data Lake	MinIO avec couches Bronze / Silver / Gold	Migration vers un stockage S3 cloud possible
Base relationnelle	PostgreSQL	SGBD modifiable
ETL	Scripts Python / Pandas reproductibles avec journalisation	Orchestration future possible

2.3. FP2 - Analyse et recommandation de livres

Cette fonction constitue le cœur métier du projet : analyser les interactions utilisateur-œuvre et produire des recommandations.

Critère	Niveau attendu	Flexibilité / évolution
Préparation ML	Matrice sparse utilisateur $\times$ œuvre	Méthodes évolutives
Périmètre du modèle	5 000 œuvres les plus notées	Seuil configurable selon les ressources
Type de recommandation	Collaborative filtering item-based	Évolution vers un modèle hybride possible
Similarité	Similarité cosinus entre œuvres	Autres métriques possibles
Recommandation par œuvre	Livres similaires à partir d'un book_key	Enrichissement par contenu possible
Personnalisation	Agrégation des similarités à partir des œuvres appréciées par l'utilisateur	Ajout de genres, descriptions et signaux supplémentaires
Nombre de recommandations	Configurable de 1 à 20 ; valeur par défaut : 5	Plage ajustable
Cold start	Absence ou faiblesse de personnalisation sans notes suffisantes	Fallback popularité / contenu à envisager

2.4. FP3 - API, exploitation et expérience utilisateur

Cette fonction rend le système accessible à une application et à l'utilisateur final.

Critère	Niveau attendu	Flexibilité / évolution
API	FastAPI avec endpoints de recherche, recommandation, création d'utilisateur et notation	Endpoints supplémentaires possibles
Recherche	Recherche par titre ou auteur dans PostgreSQL	Classement de pertinence améliorable
Interface	Streamlit	Interface remplaçable ou enrichissable
Affichage	Titre, auteur, année, éditeur, couverture et recommandations	Informations supplémentaires possibles
Utilisateurs applicatifs	Tables app_users et user_book_ratings séparées des utilisateurs historiques	Gestion de comptes plus complète possible
Ergonomie	Parcours simple : rechercher → sélectionner → recommander / noter	Personnalisation visuelle possible
Performance	Artefacts ML pré-calculés et chargés par l'application	Cache et optimisation possibles
Maintenabilité	Code modulaire, logs et documentation	Tests et monitoring à renforcer

3. Architecture technique

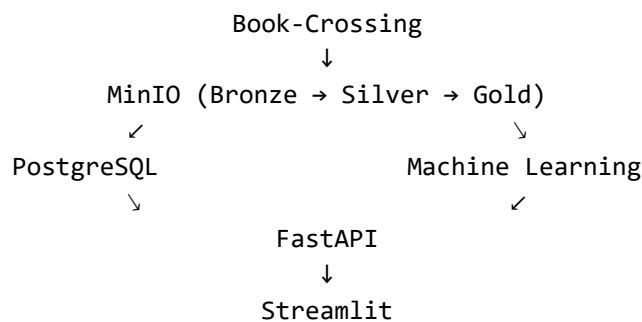
3.1. Technologies utilisées

Domaine	Technologie
Langage	Python
Traitement des données	Pandas / NumPy
Machine Learning	scikit-learn
Matrices sparse	SciPy
Sérialisation ML	joblib
Stockage objet	MinIO / S3
Base relationnelle	PostgreSQL
Accès SQL	SQLAlchemy
API REST	FastAPI
Interface utilisateur	Streamlit
Conteneurisation	Docker
Analyse exploratoire	Jupyter Notebook
Correction d'encodage	ftfy

3.2. Architecture du pipeline

1. Import des données brutes Book-Crossing.
2. Stockage Bronze dans MinIO.
3. Nettoyage et normalisation ETL.
4. Stockage Silver.
5. Création de ratings_joinable au niveau $\text{user_id} \times \text{book_key}$.
6. Construction des données analytiques et stockage Gold.
7. Chargement des tables PostgreSQL.
8. Entraînement du modèle de recommandation sur les 5 000 œuvres les plus notées.
9. Sauvegarde des artefacts similarity.joblib, book_index_map.joblib et book_metadata.joblib.
10. Exposition des fonctions via FastAPI.
11. Consommation de l'API par l'interface Streamlit.

3.3. Architecture fonctionnelle



3.4. Modèle de données applicatif

Les utilisateurs historiques du dataset et les utilisateurs réels de l'application sont séparés. Les tables `users` et `ratings` décrivent les données historiques utilisées pour l'analyse et l'apprentissage. Les tables `app_users` et `user_book_ratings` stockent les comptes et notes créés dans l'application.

4. Contraintes, limites et critères d'acceptation

4.1. Limites connues

- Dataset Book-Crossing ancien et statique.
- Métadonnées parfois hétérogènes ou incomplètes.
- book_key est construit heuristiquement : certaines traductions ou variantes peuvent rester séparées.
- Le modèle est limité aux 5 000 œuvres les plus notées.
- Cold start pour les nouveaux utilisateurs ou œuvres sans interactions suffisantes.
- Biais possible vers les œuvres populaires.
- Les couvertures reposent sur des URLs externes susceptibles de devenir indisponibles.
- Le pipeline n'est pas encore orchestré automatiquement.

4.2. Évaluation

À ce stade, l'évaluation peut être qualitative, en vérifiant la cohérence des recommandations produites. Une évaluation quantitative pourra être ajoutée avec des métriques adaptées aux systèmes de recommandation, par exemple Precision@K et Recall@K, sur un protocole de validation défini.

4.3. Critères d'acceptation

- Le pipeline produit les datasets Silver et Gold attendus sans erreur.
- Les références entre ratings, utilisateurs et œuvres sont cohérentes.
- Le modèle est entraînable et génère ses trois artefacts dans models/reco/.
- Une œuvre présente dans le modèle retourne une liste de recommandations.
- L'API permet la recherche, la recommandation, la création d'utilisateur et l'enregistrement d'une note.
- L'interface Streamlit permet de rechercher un livre, afficher ses informations et sa couverture, puis demander des recommandations.
- Un utilisateur applicatif disposant de suffisamment de notes peut recevoir des recommandations personnalisées.

5. Annexes

5.1. Organisation des données

Bronze : données CSV brutes.

Silver : données nettoyées et normalisées, notamment books_clean.parquet, users_clean.parquet, ratings_clean.parquet et ratings_joinable.parquet.

Gold : données analytiques, notamment book_popularity.parquet.

Artefacts ML : models/reco/similarity.joblib, book_index_map.joblib et book_metadata.joblib.

5.2. Endpoints principaux

- GET /books/search
- GET /recommend/book
- GET /recommend/user/{app_user_id}
- POST /users
- POST /ratings

5.3. Évolutions possibles

- Enrichissement des métadonnées via Open Library ou Google Books.
- Amélioration de la résolution des œuvres et des variantes de titres/auteurs.
- Système hybride collaborative filtering + contenu.
- Fallback de popularité pour le cold start.
- Évaluation quantitative automatisée du modèle.
- Orchestration du pipeline et automatisation du réentraînement.
- Tests automatisés, monitoring et déploiement cloud.